SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 기본

String

Python

학습시작

웹페이지에서 ☒섎ᄇᆞ☒☒ ☒묎렐☒낅땲☒☒. 라는 문구가 표시됩니다.

1. 저장할 때 [ ]로 저장하고, 읽을 때 [ ]로 읽었기 때문입니다
2. 웹페이지라면 <meta charset="[ ]"> 지정이 필요합니다

- 컴퓨터가 문자를 저장하는 방법을 배울 수 있어요.
- 언어에 따라 문자열을 다루는 방법이 어떻게 다른지 이해할 수 있게 됩니다.
- 문자 인코딩 방식이 다를 때 발생하는 문제와 해결 방법을 이해할 수 있어요.

학습목표

- 컴퓨터에서 문자를 표현하는 방법에 대해 이해한다.
- 컴퓨터에서 문자열을 저장하는 방식을 이해하고, 이를 활용할 수 있다.
- 문자열을 조작하는 방법을 이해하고 이를 활용할 수 있다.
- 패턴 매칭에 대해 이해하고, 설명할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

문자열

- 코드체계
- 문자열
- 연습문제
- 연산

패턴 매칭

- 고지식한 패턴 검색
- KMP 알고리즘
- 보이어-무어 알고리즘
- 문자열 암호화

문자열

코드체계

컴퓨터에서의 문자표현

코드체계

문자에 대응되는 숫자를 정한 것입니다.

예를 들어 글자 A를 메모리에 저장하는 방법을 생각해 볼까요?

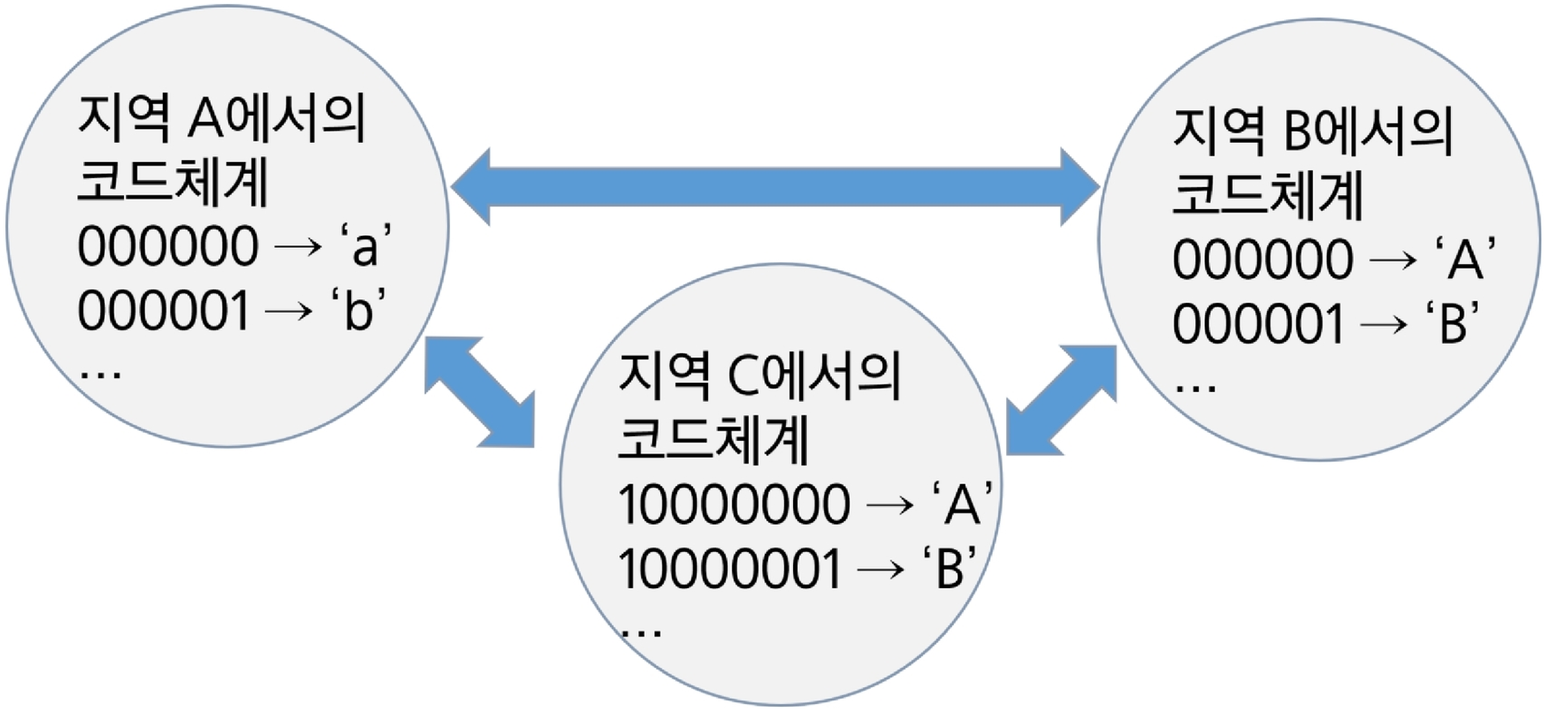
메모리는 숫자만을 저장할 수 있습니다.

영어가 대소문자 합쳐서 52자 이므로 6비트(64가지 경우)면 저장할 수 있습니다.

예를 들어 ‘a’ -> 000000(0) , ‘b’ -> 000001(1), ‘c’ -> 000010(2) 같이 6비트 이진수(10진수)를 배정합니다.

코드체계의 문제

- 네트워크가 발전되기 전 미국의 각 지역 별로 코드체계를 정해 놓고 사용.
- 네트워크가 발전하면서 서로 정보를 주고 받을 때 정보를 달리 해석한다는 문제가 생김.



<그림1_SW문제해결 기본_String_지역별 코드체계>

코드체계의 개선

- 그래서 혼동을 피하기 위해 표준안을 만들기로 함.
- 1967년, 미국에서 ASCII(American Standard Code for Information Interchange)라는 문자 인코딩 표준이 제정됨.
- ASCII는 7-bit 인코딩으로 128문자를 표현하며, 33개의 출력 불가능한 제어 문자들과 공백을 비롯한 95개의 출력 가능한 문자들로 이루어짐.

출력 가능 아스키 문자 (32~126)

ASCII	문자	ASCII	문자	ASCII	문자	ASCII	문자	ASCII	문자	ASCII	문자
32		48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	₩	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o		

<표1_SW문제해결 기본_String_출력 가능 아스키 문자>

확장 아스키 (Extended ASCII)

- 표준 문자 이외의 악센트 문자, 도형 문자, 특수 문자, 특수 기호 등 부가적인 문자를 128개 추가.
- 표준 아스키는 7-bit를 사용하여 문자를 표현, 확장 아스키는 1Byte 내의 8-bit를 모두 사용함으로써 추가적인 문자를 표현할 수 있음.
- 컴퓨터 생산자와 소프트웨어 개발자가 여러 가지 다양한 문자에 할당할 수 있도록 하고 있고, 이렇게 할당된 확장 부호는 표준 아스키와 같이 서로 다른 프로그램이나 컴퓨터 사이에 교환되지 못함.
- 표준 아스키는 마이크로컴퓨터 하드웨어 및 소프트웨어 사이에서 세계적으로 통용되는 데 비해, 확장 아스키는 프로그램이나 컴퓨터 또는 프린터가 그것을 해독할 수 있도록 설계되어 있어야만 올바르게 해독될 수 있음.

확장 아스키 예

ASCII	문자	ASCII	문자	ASCII	문자	ASCII	문자	ASCII	문자	ASCII	문자
161	¡	171	«	181	µ	191	¿	201	É	211	Ó
162	¢	172	¬	182	¶	192	À	202	Ê	212	Ô
163	£	173		183	·	193	Á	203	Ë	213	Õ
164	¤	174	®	184	¸	194	Â	204	Ì	214	Ö
165	¥	175	¯	185	¹	195	Ã	205	Í	215	×
166	¦	176	°	186	º	196	Ä	206	Î	216	Ø
167	§	177	±	187	»	197	Å	207	Ï	217	Ù
168	¨	178	²	188	¼	198	Æ	208	Ð	218	Ú
169	©	179	³	189	½	199	Ç	209	Ñ	219	Û
170	ª	180	´	190	¾	200	È	210	Ò	220	Ü

<표2_SW문제해결 기본_String_확장 아스키 예>

유니코드

- 컴퓨터가 발전하면서 미국뿐 아니라 각 나라에서 컴퓨터가 발전함.
- 각 국가들은 자국의 문자를 표현하기 위하여 코드 체계를 만들어서 사용하게 됨.
  - 우리나라도 한글 코드체계를 만들어 사용했고 조합형, 완성형 두 종류를 가지고 있었습니다.
- 인터넷이 전 세계로 발전하면서 ASCII를 만들기 전과 같은 문제가 국가 사이에 정보를 주고받을 때 발생함.
- 자국의 코드체계를 타 국가가 가지고 있지 않으면 정보를 잘못 해석할 수밖에 없음.
- 다국어 처리를 위해 표준인 유니코드를 만듦.
  - 비영리 단체인 유니코드 컨소시엄(https://home.unicode.org)에서 관리합니다.
  - 이모지(Emoji)도 유니코드 문자입니다.

유니코드 :
다국어 처리를 위한 표준 코드체계이다.

<개념1_SW문제해결 기본_String_유니코드>

유니코드의 일부

- 출처 : Unicode Association, The Unicode Standard, Version 16.0

	000	001	002	003	004	005	006	007
0	NUL 0000	DLE 0010	SP 0020	0 0030	@ 0040	P 0050	` 0060	p 0070
1	SOH 0001	DC1 0011	! 0021	1 0031	A 0041	Q 0051	a 0061	q 0071
2	STX 0002	DC2 0012	" 0022	2 0032	B 0042	R 0052	b 0062	r 0072
3	ETX 0003	DC3 0013	# 0023	3 0033	C 0043	S 0053	c 0063	s 0073
4	EOT 0004	DC4 0014	$ 0024	4 0034	D 0044	T 0054	d 0064	t 0074

	B30	B31	B32	B33	B34	B35	B36	B37
0	대 B300	댐 B310	댠 B320	댰 B330	댈 B340	댴 B350	덁 B360	데 B370
1	댁 B301	댑 B311	댡 B321	당 B331	댉 B341	댵 B351	덡 B361	덱 B371
2	댂 B302	댒 B312	댢 B322	댲 B332	댊 B342	댶 B352	덢 B362	덲 B372
3	댃 B303	댓 B313	댣 B323	댳 B333	댋 B343	댷 B353	덣 B363	덳 B373
4	댄 B304	댔 B314	달 B324	닼 B334	댍 B344	더 B354	덤 B364	덴 B374

<그림2_SW문제해결 기본_String_유니코드의 일부>

TIP

- ‘A’의 유니코드는 16진수(10진수)로 0041(65) 입니다.
- ‘덤’의 유니코드는 16진수로 B364 입니다.
- print(‘₩uB364’)로 ‘덤‘ 을 출력할 수 있습니다.

유니코드 Character Set

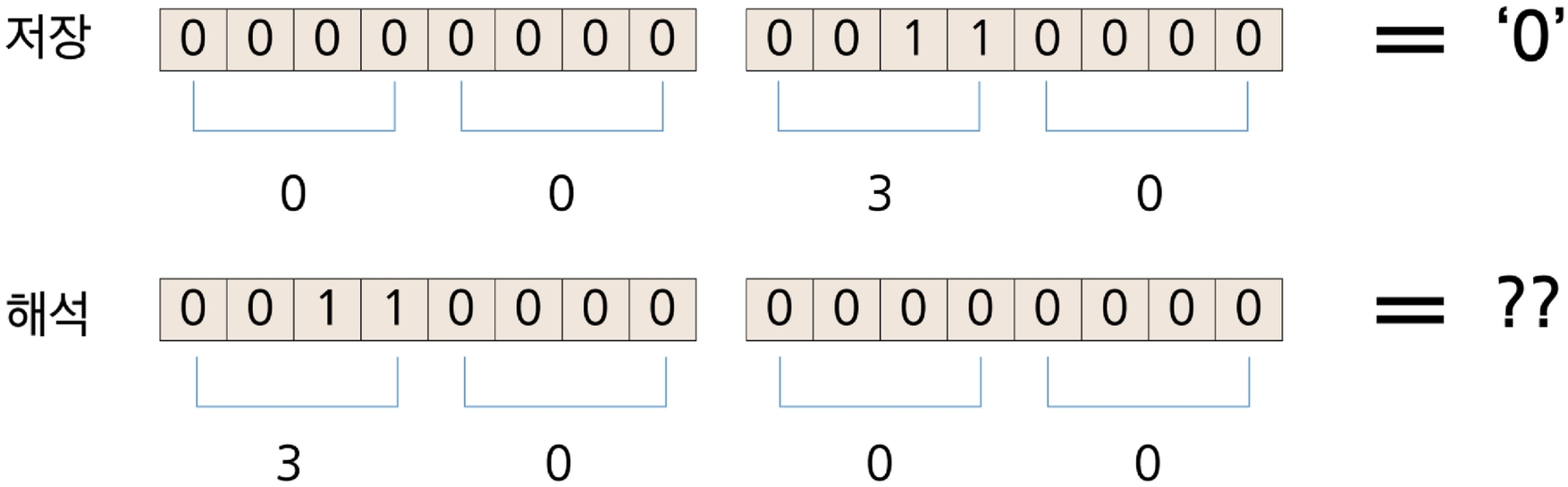
- 유니코드도 다시 Character Set으로 분류됨.
- UCS-2(Universal Character Set 2)
- UCS-4(Universal Character Set 4)
- 유니코드를 저장하는 변수의 크기를 정의함.
  - 그러나, 바이트 순서에 대해서는 표준화 하지 못했습니다.
- 파일을 일을 때 UCS-2, UCS-4인지 인식하고 각 경우를 구분해서 모두 다르게 구현해야 하는 문제가 발생.
  - 그래서 유니코드의 적당한 외부 인코딩이 필요하게 되었습니다.

바이트 단위 저장 순서

little-endian과 big-endian :
여러 바이트로 표현된 수를 바이트 단위로 저장하는 순서를 나타낸다.

<개념2_SW문제해결 기본_String_endian>

- 바이트 단위 저장 순서가 정해지지 않은 경우 잘못된 해석 가능성
  - 여러 바이트로 이루어진 데이터를 저장하는 방식을 Endian 이라고 합니다.
  - Big-endian은 상위 바이트(MSB, Most Significant Byte) 를 가장 낮은 주소에 저장합니다.
  - Little-endian은 하위 바이트(LSB, Least Significant Byte) 를 가장 낮은 주소에 저장합니다.



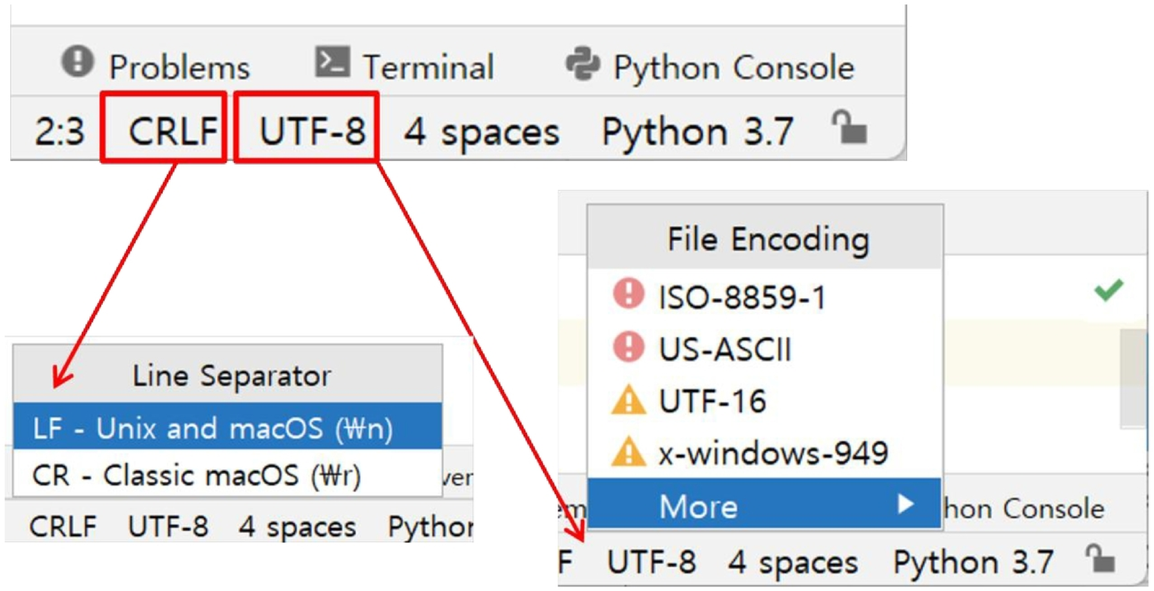
<그림3_SW문제해결 기본_String_바이트 단위 저장 순서>

유니코드 인코딩 (UTF : Unicode Transformation Format) (일부)

- UTF-8 (in web)
  - MIN : 8-bit, MAX: 32-bit(1 Byte * 4)
  - 필요한 크기에 따른 저장 방법 예
    - 0xxxxxxx
    - 110xxxxx 10xxxxxx
    - 1110xxxx 10xxxxxx 10xxxxxx
    - 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
- UTF-16 (in windows, java)
  - MIN : 16-bit, MAX: 32-bit(2 Byte * 2)
- UTF-32 (in unix)
  - MIN : 32-bit, MAX: 32-bit(4 Byte * 1)

인코딩의 예

- PyCharm에서 인코딩 방식을 확인하는 방법.



<그림4_SW문제해결 기본_String_PyCharm의 인코딩>

TIP

- 인코딩과 상관없지만 OS에 따라 줄 바꿈을 표시하는 문자(코드)가 다릅니다.
- 윈도우는 CR(13)과 LF(10) 두 글자를 사용합니다.
- Unix와 macOS는 LF(10) 한 글자를 사용합니다.

인코딩에 따른 text 파일의 차이

- text.txt에 다음과 같이 저장.

```
ABC
한글
```

- 인코딩 방식을 바꿔가며 저장된 파일을 16진수 편집기(Hexadecimal Editor)로 읽기.

UTF-8

UTF-16

<그림5_SW문제해결 기본_String_인코딩에 따른 text 파일의 차이 >

출처 : https://hexed.it 화면 캡쳐

Python 인코딩

- UTF-8
  - 2.x 버전은 기본 인코딩 방식이 ASCII로 UTF-8 사용시 코드 첫 줄에 다음 문장 추가해야 함.
    - #-*- coding: utf-8 -*-
  - 3.x 버전은 UTF-8 방식이 기본이므로 표시 생략.
- 다른 인코딩 방식으로 처리 시 첫 줄에 작성하는 항목에 원하는 인코딩 방식을 지정해주면 됨.

코드체계의 핵심

코드체계 :
컴퓨터가 처리할 수 있도록 문자에 고유한 숫자를 부여하는 것이다.

<개념3_SW문제해결 기본_String_코드체계>

코드체계의 핵심

전 세계의 모든 문자를 컴퓨터가 일관되게 표현하고 처리하는 것.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

문자열

문자열이란?

문자열 :
문자들이 순서대로 나열된 데이터로,
텍스트 표현이나 처리에 사용된다.

<개념4_SW문제해결 기본_String_문자열>

문자열 (String)

문자들이 순서대로 나열된 데이터

문자를 차례대로 모아 놓은 것으로,
텍스트(문장, 단어, 기호 등)를 표현하거나 처리할 때 사용됩니다.

문자열의 분류

- Length-Controlled 문자열
  - 문자열의 길이 정보를 함께 저장해서, 그 길이만큼 문자 데이터를 읽는 방식입니다.
  - Java, Python, 네트워크 패킷에 사용됩니다.

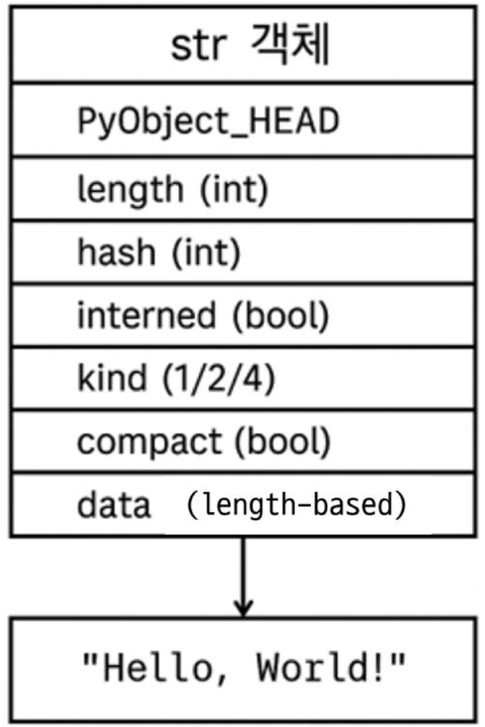
- Delimited 문자열
  - 문자열의 끝을 나타내는 특정한 구분자(Delimiter)가 있어서, 구분자가 나올 때까지 문자열로 인식합니다.
  - C언어는 널문자(`null, '\0'`)를 사용합니다.

Length-Controlled	5	‘H’	‘e’	‘l’	‘l’	‘o’
Delimited	‘H’	‘e’	‘l’	‘l’	‘o’	‘₩0’

<그림6_SW문제해결 기본_String_문자열의 저장 방식>

파이썬 str 클래스 구조

- 길이 외에 다른 정보도 저장
  - PyObject_HEAD: 모든 Python 객체가 상속하는 공통 구조입니다.
  - length: 문자열의 길이 입니다.
  - hash: 문자열의 해시값으로, 딕셔너리 키로 쓸 때 사용됩니다.
  - interned: 같은 문자열을 관리하는 플래그입니다.
  - kind: 문자열 인코딩의 크기입니다.
  - data: 문자열이 저장된 실제 메모리 주소를 가리키는 포인터입니다.



<그림7_SW문제해결 기본_String_str 클래스 구조>

C언어에서 문자열

- 문자열은 문자들의 배열 형태로 구현된 응용 자료형.

- 문자배열에 문자열을 저장할 때는 항상 마지막에 끝을 표시하는 널문자('₩0') 필요.
  - char ary[ ]="abc"; // char ary[ ]={'a', 'b', 'c', '₩0'};

- 문자열 처리에 필요한 연산을 함수 형태로 제공함.
  - strlen(), strcpy(), strcmp(),…

Java에서의 문자열

- 문자열 데이터를 저장, 처리해주는 클래스를 제공함.

- String 클래스.
  - String str="abc"; //또는 String str = new String("abc");

- 문자열 처리에 필요한 연산을 연산자, 메소드 형태로 제공함.
  - +, length(), replace(), split(), substring(),…

Python3에서의 문자열

- 텍스트 데이터의 취급 방법이 통일되어 있음
  - python2와 달리 바이트 문자열과 Unicode 구분이 없습니다.
  - 유니코드(Unicode) 기반이라 어떤 언어나 기호도 동일한 방식으로 처리합니다.
- 문자열 기호.
  - ‘(홑따옴표), “(쌍따옴표), ‘‘‘ (홑따옴표 3개), “““ (쌍따옴표 3개)
- 연산
  - + 연결(Concatenation)
    - 문자열 + 문자열 : 이어 붙여주는 역할
    - 예) 'ab' + 'c'
  - * 반복
    - 문자열 * 수 : 수만큼 문자열이 반복
    - 'ab' * 3

Python3에서의 문자열

- 문자열은 데이터의 순서가 구분되는 시퀀스 자료형으로 분류됨.
  - 시퀀스 자료형에서 사용할 수 있는 인덱싱, 슬라이싱 연산들을 사용할 수 있습니다.
- 문자열 클래스에서 제공되는 메소드.
  - replace(), split(), isalpha(), find()
- 문자열은 튜플과 같이 요소값을 변경 할 수 없음(immutable).

1 인덱스를 사용할 수 있습니다.

```
s = 'abc'
print(s[1])
```

2 요소값을 변경할 수 없습니다.

```
s = 'abc'
s[1] = 'a'  // TypeError: 'str' object does not support item assignment
```

C, Java, Python3 문자열의 차이

- C는 아스키 코드로 저장
  - 한글을 출력할 수 있으나 콘솔의 도움을 받아야 합니다.
- Java는 유니코드(UTF=16, 2-Byte)로 저장
- Python3는 유니코드(UTF-8)로 저장

C

```
char * name = “홍길동";
int count = strlen(name);
printf(“%d”, count);
```

6이 출력된다.

Java

```
String name = “홍길동";
System.out.println(name.length());
```

3이 출력된다.

Python3

```
name = "홍길동"
print(len(name))
```

3이 출력된다.

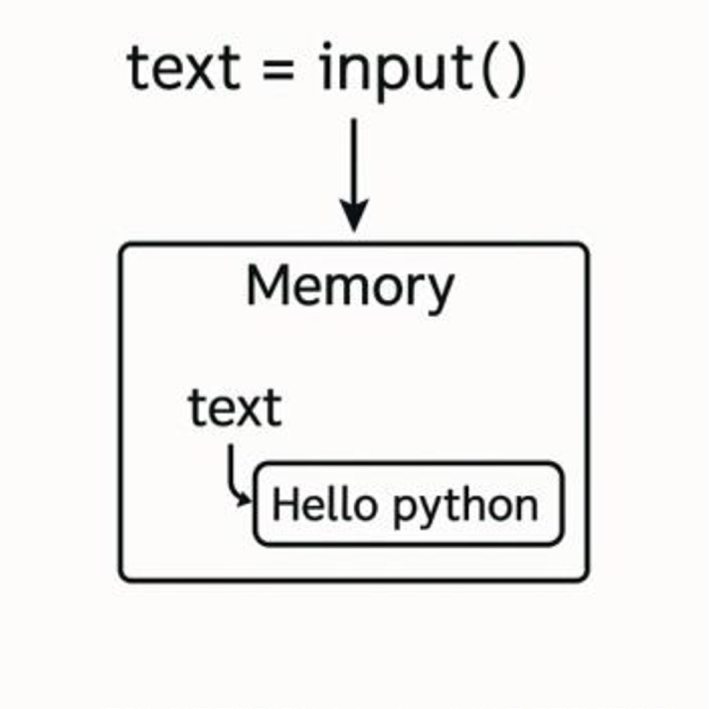
<그림8_SW문제해결 기본_String_언어별 문자열 길이 판단>

Python에서의 문자열 입력

- input() 함수로 읽기

```
text = input()
```

- Hello python을 입력하면 빈칸을 포함해 한 행을 읽어 들입니다.



<그림9_SW문제해결 기본_String_input() 함수로 문자열 입력하기>

TIP

- text = list(input())을 사용하면 각 글자가 리스트의 원소로 저장됩니다.
- ['H', 'e', 'l', 'l', 'o', ' ', 'p', 'y', 't', 'h', 'o', 'n']
- 원소 별로 다른 글자로 바꿀 수 있습니다.

연습문제

Python에서의 문자열 입력

- 두 개의 문자열 s1과 s2. s1의 각 글자가 s2에 모두 존재하는가?

XYPV
EOGGXYPVSY

```
s1 = input()
s2 = input()
```

```
s1 = list(input())
s2 = list(input())
```

Python에서의 문자열 입력

- 첫 줄에 N, 다음에 NxN 문자열. 'Z'가 존재하는가?

```
5
GOFFA
OYECR
UJAJQ
JAEZN
WJAKC
```

```
N = int(input())
text = [input() for _ in range(N)]
```

```
N = int(input())
text = [list(input()) for _ in range(N)]
```

Python에서의 문자열 입력

- 첫 줄에 N, 다음에 NxN 지도, # 집중호우 피해구역. 피해구역 수는?

```
5
000#0
00###
000#0
00#00
000#0
```

```
N = int(input())
text = [input() for _ in range(N)]
```

```
N = int(input())
text = [list(input()) for _ in range(N)]
```

Python에서의 문자열 입력

- 첫 줄에 N, 다음에 NxN 문자열. AB / CD 패턴이 존재하는가?

```
5
GOFFA
OYECR
UJAJQ
JAEZN
WJAKC
```

```
N = int(input())
text = [input() for _ in range(N)]
```

```
N = int(input())
text = [list(input()) for _ in range(N)]
```

Python에서의 문자열 입력

- 첫 줄에 N, 다음에 NxN 미로. 0 통로, 1 벽, # 로봇. 상하좌우로 모두 이동할 수 있는 칸 수는?

```
5
00101
1#101
00000
11010
00010
```

```
N = int(input())
text = [input() for _ in range(N)]
```

```
N = int(input())
text = [list(input()) for _ in range(N)]
```

연산

문자열 뒤집기

- 문자열을 역순으로 재정의

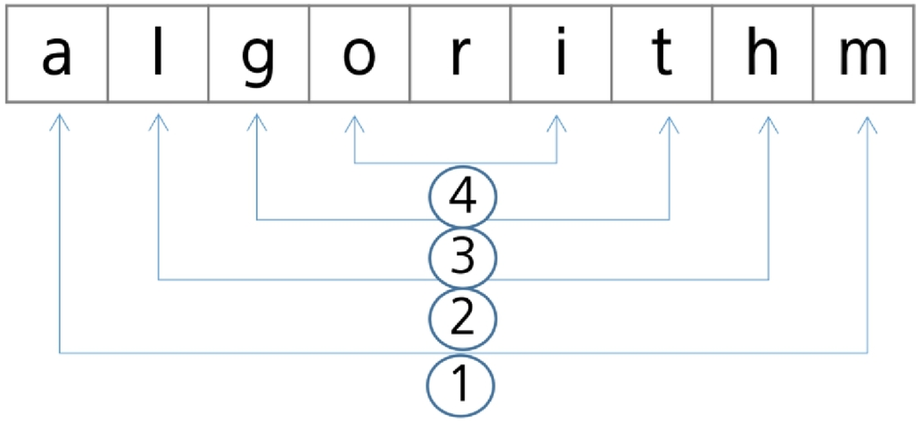
```
s = 'Reverse this strings'     #   'sgnirts siht esreveR'
s = s[::-1]
```

- 리스트로 변환 후 다시 문자열로 변환

```
s = 'abcd’
s = list(s)
s.reverse()
s = ''.join(s)
```

회문

- "기러기", "토마토", "스위스"와 같이 똑바로 읽어도 거꾸로 읽어도 똑같은 문장이나 낱말
  - 문자열 길이의 반만 비교하면 됩니다.



문자열 길이 9
9 / 2 = 4.5
4회 반복

<그림10_SW문제해결 기본_String_회문>

```
is_palindrome(txt):
        for i : 0 -> len(txt)/2 - 1
                if txt[i] != txt[len(txt) – 1 - i]
                        return False
        return True
```

문자열 비교

- == 연산자와 is 연산자

- ==는 값(value)이 같은지를 비교합니다.
- is는 객체의 정체성(identity), 즉 **같은 객체(메모리 주소)**인지 비교합니다.
- == 연산자는 내부적으로 특수 메서드 __eq__( )를 호출합니다.

```
s1 = 'abc'
s2 = 'abc'
s3 = 'def'
s4 = s1
s5 = s1[:2] + 'c'

print(s1 == s2)          # True
print(s1 is s2)          # True
print(s4 == s5)          # True
print(s4 is s5)          # False
```

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)  # True  (값이 같음)
print(a is b)  # False (다른 객체임)
```

C와 Java에서의 문자열 비교

- C
  - strcmp() 함수를 사용해 문자열의 내용을 비교합니다.

- Java
  - == 연산자는 객체의 주소(참조)를 비교합니다. (Python의 is 연산자 역할)
  - equals() 메소드는 객체의 내용을 비교합니다. (Python의 == 역할)

사전 순서 비교

- 비교 연산자 < 사용
  - 유니코드를 비교합니다.

1 두 문자열의 사전상 순서를 비교하는 my_strcmp() 함수를 구현한 예 입니다.

```
def my_strcmp(s1, s2):
    if s1 < s2:
        return -1                # s1이 s1보다 사전 순서상 앞서는 경우
    elif s1 > s2:
        return 1                 # s1이 s2보다 사전 순서당 나중인 경우
    else:
        return 0                 # s1과 s2 문자열의 내용이 같은 경우
```

TIP

- 문자열에 대한 < 연산은 유니코드 값에 대한 비교입니다.
- 따라서 'Apple' < 'apple' 도 True 이고 'Zebra' < 'apple' 도 True입니다.

문자열 숫자를 숫자로 변환

1 문자열을 숫자로 변환하는 예 입니다.

```
a = int('123')
b = float('3.14')
c = int('A0', 16)       # 문자열 'A0'를 16진법으로 해석해서 변환
```

2 숫자를 문자열로 변환하는 예입니다.

```
a = str(123)
b = str(3.14)
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

패턴 매칭

SAMSUNG SW AI ACADEMY FOR YOUTH

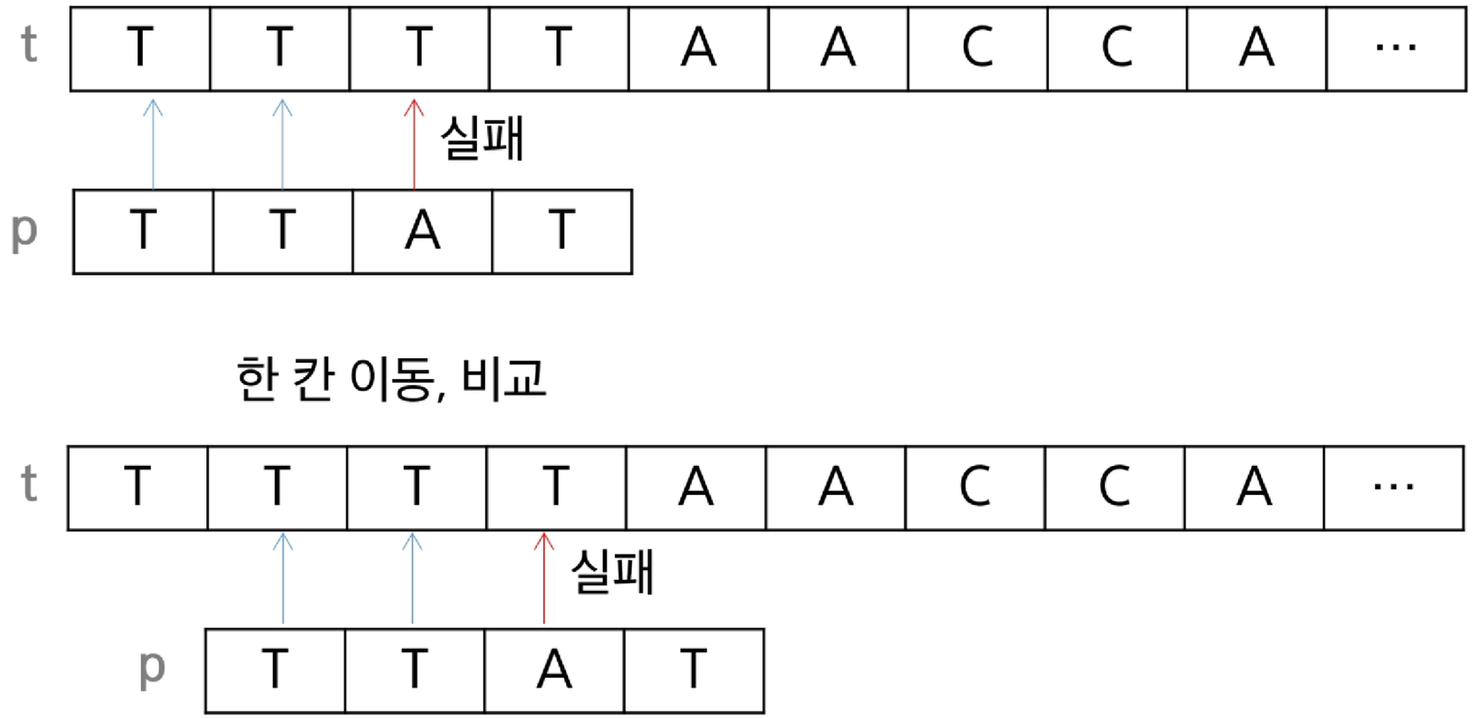
함께가요 미래로!
Enabling People

고지식한 패턴 검색

고지식한 알고리즘 (Brute Force)

- 단순한 방법

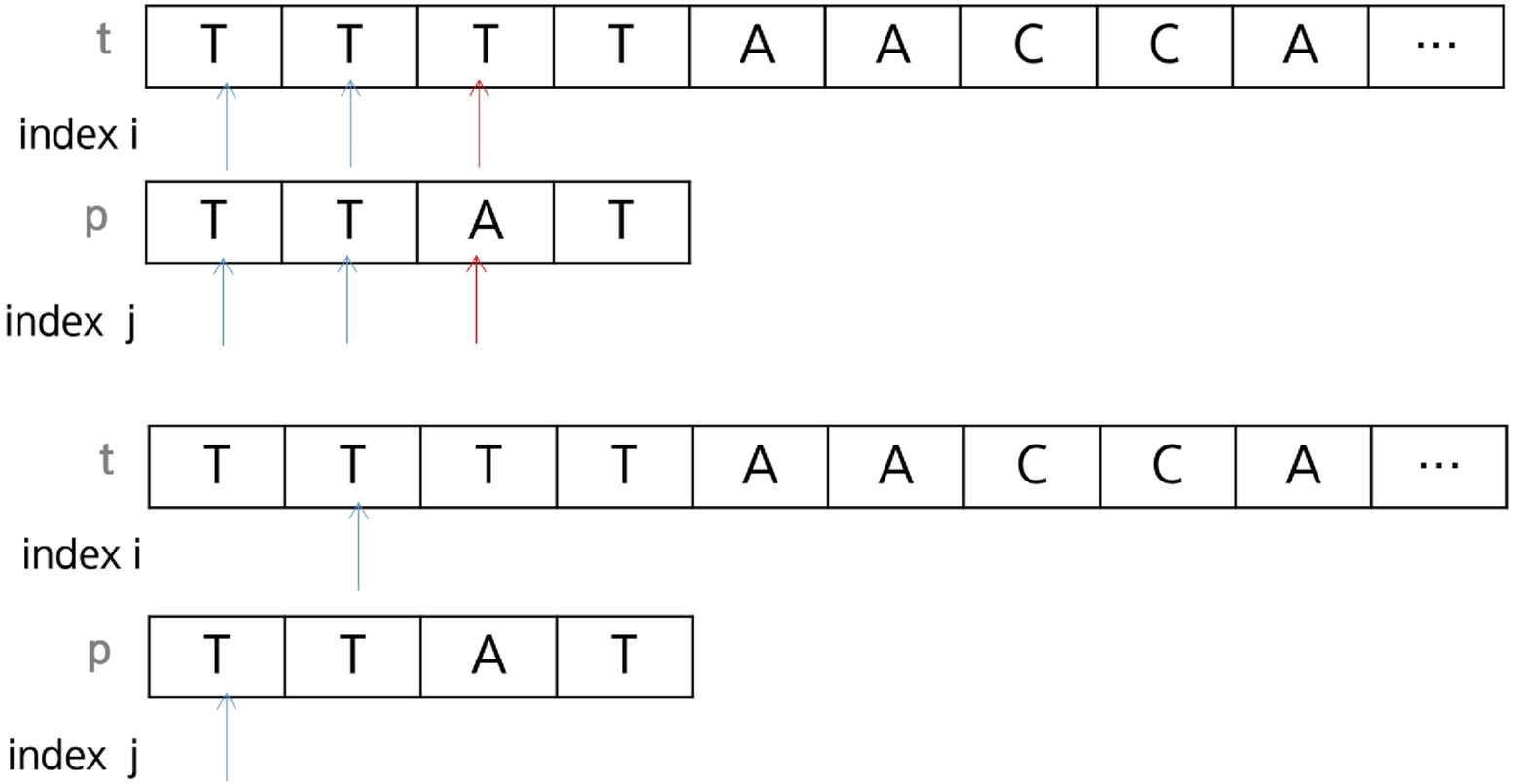
- 본문 문자열을 처음부터 끝까지 차례대로 순회하면서 패턴 내의 문자들을 일일이 비교하는 방식입니다.



<그림11_SW문제해결 기본_String_고지식한 패턴 매칭>

구현

- 본문의 비교위치 i, 패턴에서의 비교위치 j
  - 일치하면 i, j 증가, 실패하면 i는 이전 시작위치 다음으로, j는 맨 앞으로 초기화



<그림12_SW문제해결 기본_String_고지식한 패턴 매칭 첫 번째 위치>

코드

- 3_string/pattern.py 참고

```
def brute_force(p, t) : # p 찾을 패턴, t 본문 문자열, 패턴이 있으면 인덱스, 없으면 -1 리턴
    i = 0 # t의 인덱스
    j = 0 # p의 인덱스
    M = len(p)
    N = len(t)
    while j < M and i < N :
        if t[i] != p[j] :       # 다른 글자인 경우
            i = i - j
            j = -1
        i = i + 1
        j = j + 1
    if j == M : return i - M    # 검색 성공
    else : return -1                    # 검색 실패
```

시간 복잡도

- 최악의 경우 텍스트의 모든 위치에서 패턴을 비교해야 하므로 O(MN)이 됨.
- 길이가 10,000인 문자열에서 길이 80인 패턴을 찾는 경우.
  - 최악의 경우 약 10,000*80 = 800,000 번의 비교가 필요합니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

KMP 알고리즘

KMP 알고리즘

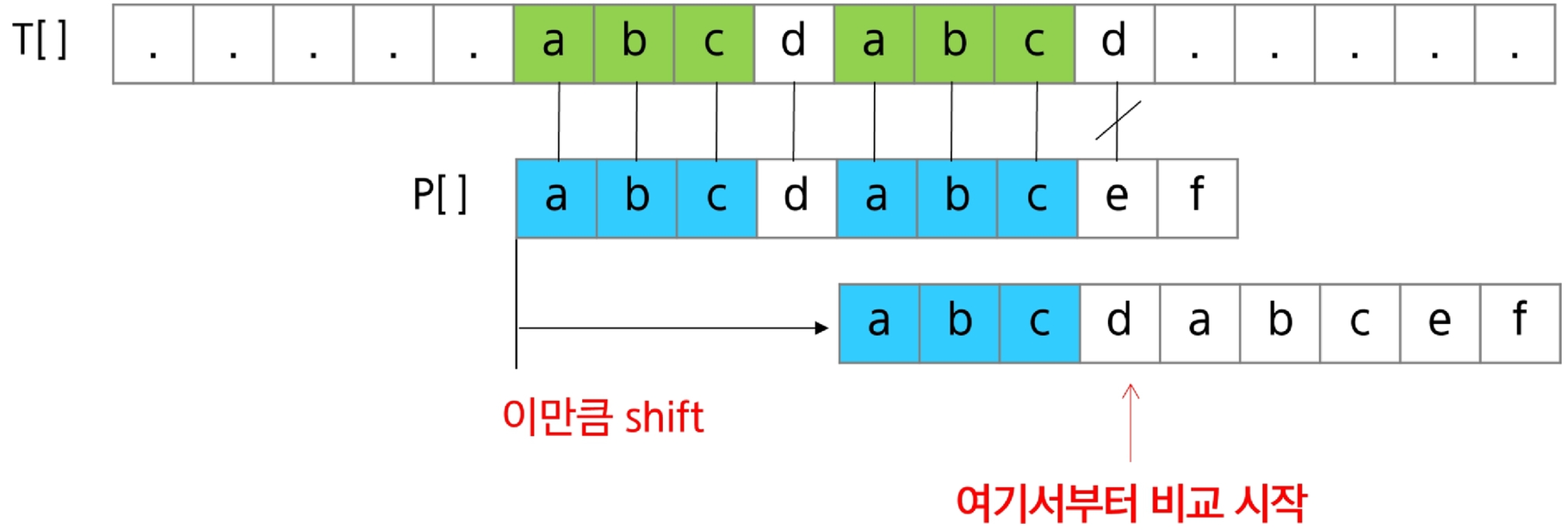
- 연구자인 Knuth, Morris, Pratt 세 사람의 이름에서 유래

- 패턴의 각 위치에서 매칭에 실패했을 때 돌아갈 위치를 미리 계산
  - 불일치가 발생한 글자의 앞 부분에 어떤 문자가 있는지를 미리 알고 있게 됩니다.
  - 조건에 따라, 불일치가 발생한 앞 부분에 대하여 다시 비교하지 않을 수 있습니다.
  - 불일치가 발생했을 경우 이동할 다음 위치를 계산하는 전처리가 필요합니다.

- 시간 복잡도
  - 패턴의 길이가 M일 때 전처리에 걸리는 시간은 O(M) 입니다.
  - 텍스트의 길이가 N일 때 검색은 최악의 경우 O(N) 입니다.
  - 결과적으로 O(M+N)이 됩니다.
  - 만약 M이 고정된 값으로 매우 짧다면 평균적으로 Θ(N)이 됩니다.

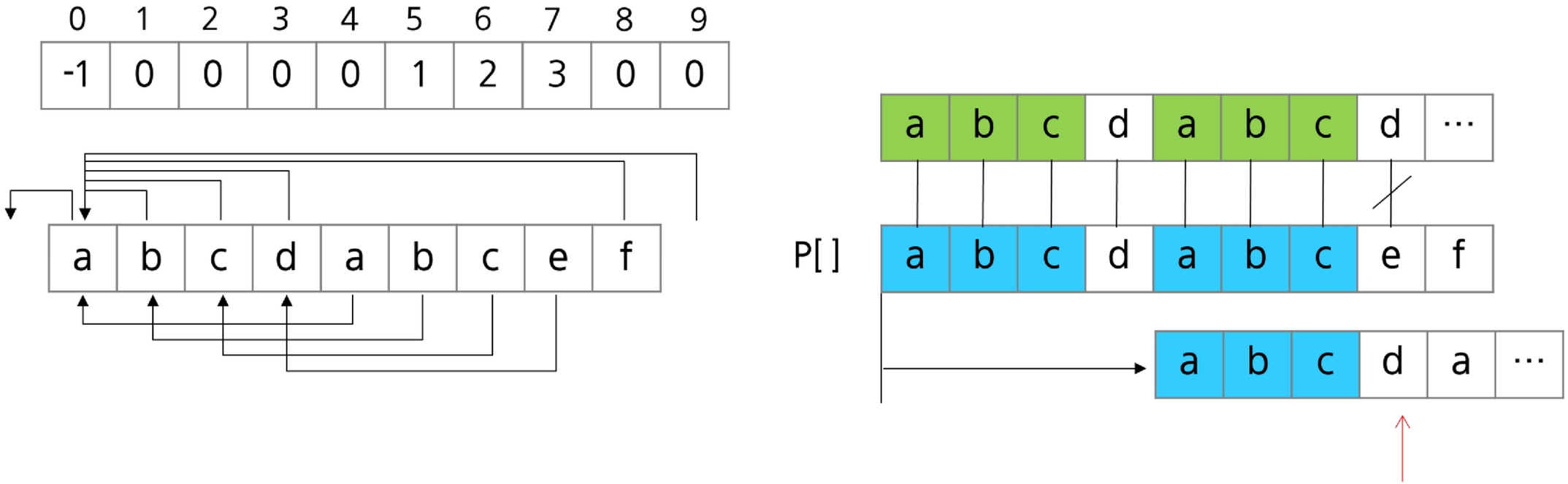
아이디어

- 텍스트에서 abcdabc까지는 매치되고, e에서 실패한 상황
  - 패턴의 맨 앞의 abc와 실패 직전의 abc는 동일함을 이용할 수 있습니다.



LPS(Longest Prefix which is also Suffix) 배열

- 접두사이자 접미사인 문자열의 최대 길이
- 일치하지 않을 때 비교할 인덱스
- 구현에 따라 next, pi 배열로도 불림



<그림14_SW문제해결 기본_String_KMP LPS 배열>

SAMSUNG SW AI ACADEMY FOR YOUTH

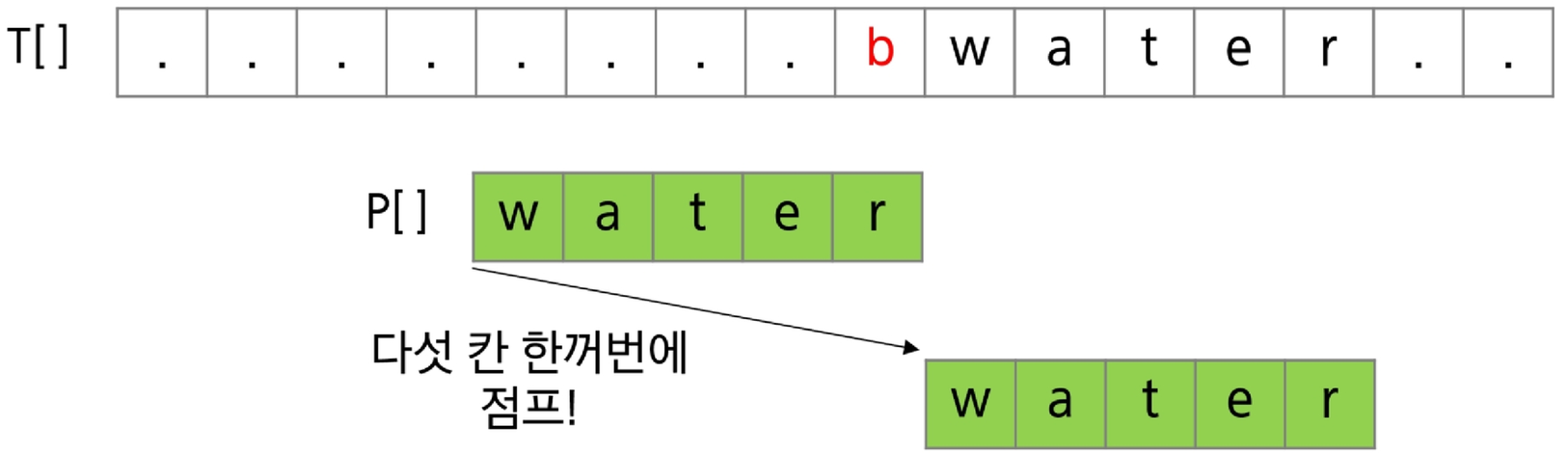
함께가요 미래로!
Enabling People

보이어-무어 알고리즘

보이어-무어(Boyer-Moore) 알고리즘

- 상용 소프트웨어에서 채택하고 있는 알고리즘
- 실제 검색에서 매우 빠른 성능

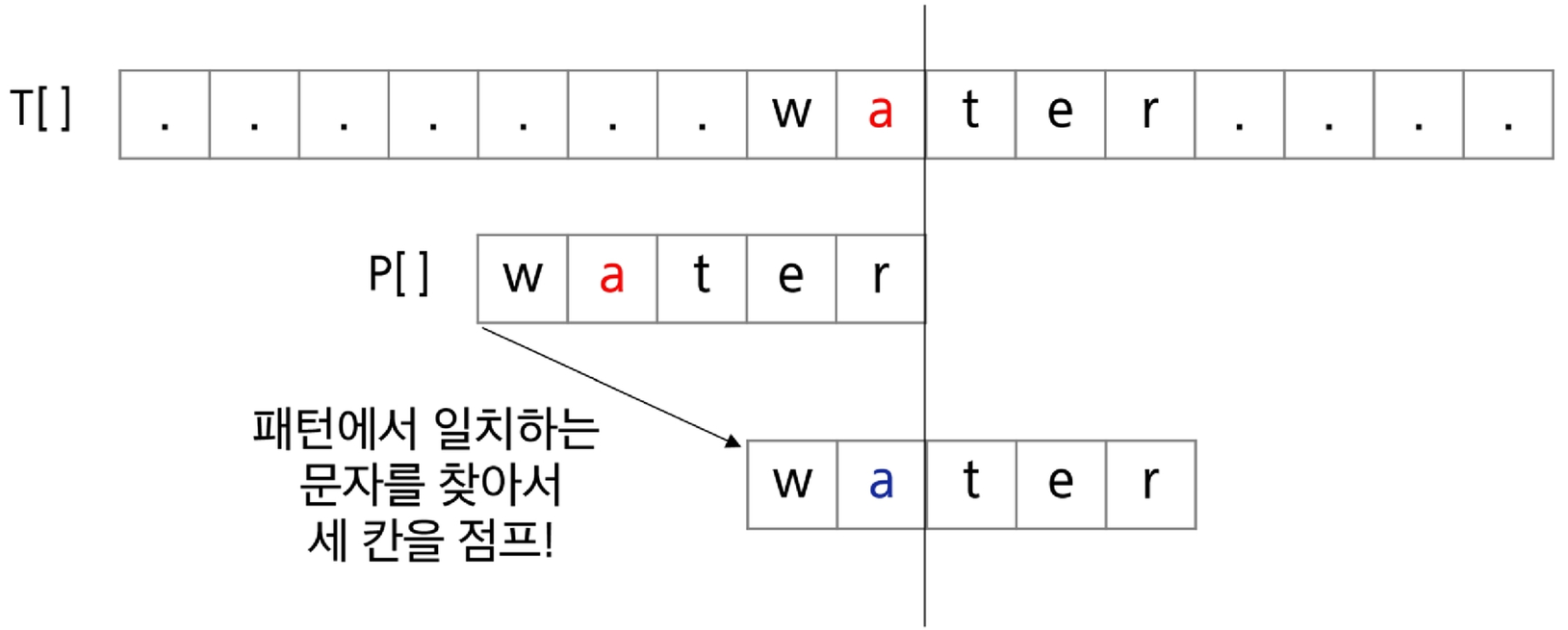
- 패턴 오른쪽 끝에 있는 문자가 불일치하고 이 문자가 패턴 내에 존재하지 않는 경우, 이동 거리는 패턴의 길이만큼이 됩니다.



<그림15_SW문제해결 기본_String_불일치 문자가 패턴에 없는 경우>

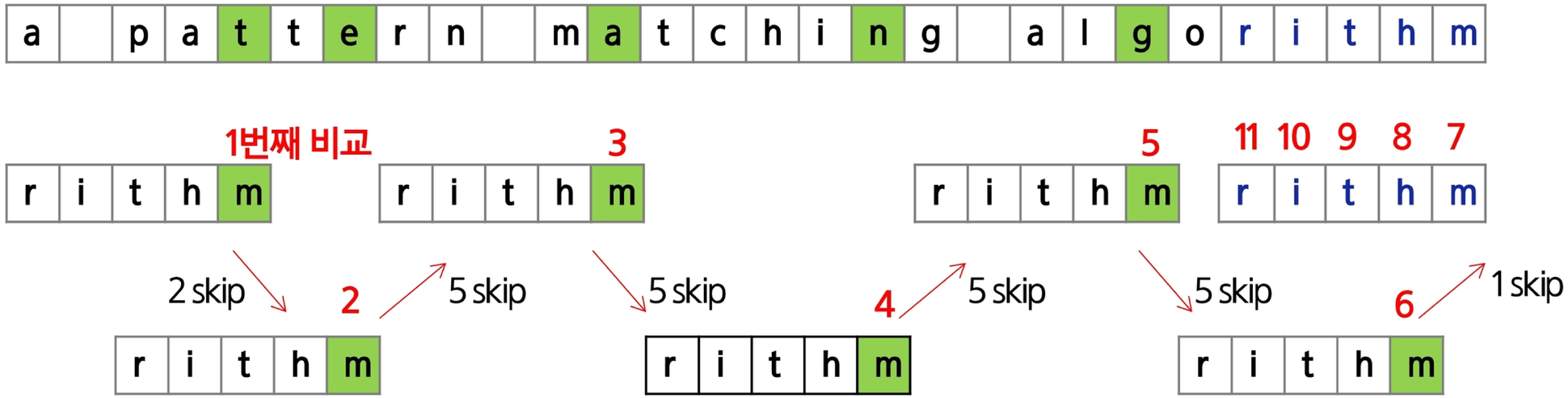
불일치 문자 휴리스틱(Bad-Character Heuristic)

- 오른쪽 끝에 있는 문자가 불일치 하고 이 문자가 패턴 내에 존재할 경우
  - 불일치 문자 오른쪽에서 일치한 부분을 고려해 점프합니다.



<그림16_SW문제해결 기본_String_불일치 문자가 패턴에 있는 경우>

Skip 배열



Skip 배열

r	i	t	h	m	다른 모든 문자
4	3	2	1	0	5

<그림17_SW문제해결 기본_String_Skip 배열>

문자열 매칭 알고리즘 비교

- 찾고자 하는 문자열 패턴의 길이 m, 총 문자열 길이 n
- 고지식한 패턴 검색 알고리즘 : 수행시간 O(mn)
- 카프-라빈 알고리즘 : 수행시간 Θ(n)
- KMP 알고리즘 : 수행시간 Θ(n)
- 보이어-무어 알고리즘
  - 앞의 두 매칭 알고리즘들은 텍스트 문자열의 문자를 적어도 한 번씩 훑기 때문에 최선의 경우에도 Ω(n)입니다.
  - 보이어-무어 알고리즘은 텍스트 문자를 다 보지 않아도 됩니다.
  - 패턴의 오른쪽부터 비교합니다.
  - 완전한 보이어-무어 알고리즘은 일치 접미부 휴리스틱을 함께 사용합니다.
  - 최악의 경우 O(mn)이지만 일반적으로 Θ(n)보다 시간이 덜 듭니다.
  - grep, VS Code, JVM 등에서 사용된다고 알려져 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

문자열 암호화

시저 암호(Caesar cipher)

- 줄리어스 시저가 사용했다고 하는 암호
- 시저는 기원전 100년경에 로마에서 활약했던 장군
- 시저 암호에서는 평문에서 사용되고 있는 알파벳을 일정한 문자 수만큼 [평행이동] 시킴으로써 암호화
- 1 만큼 평행한 암호화의 예

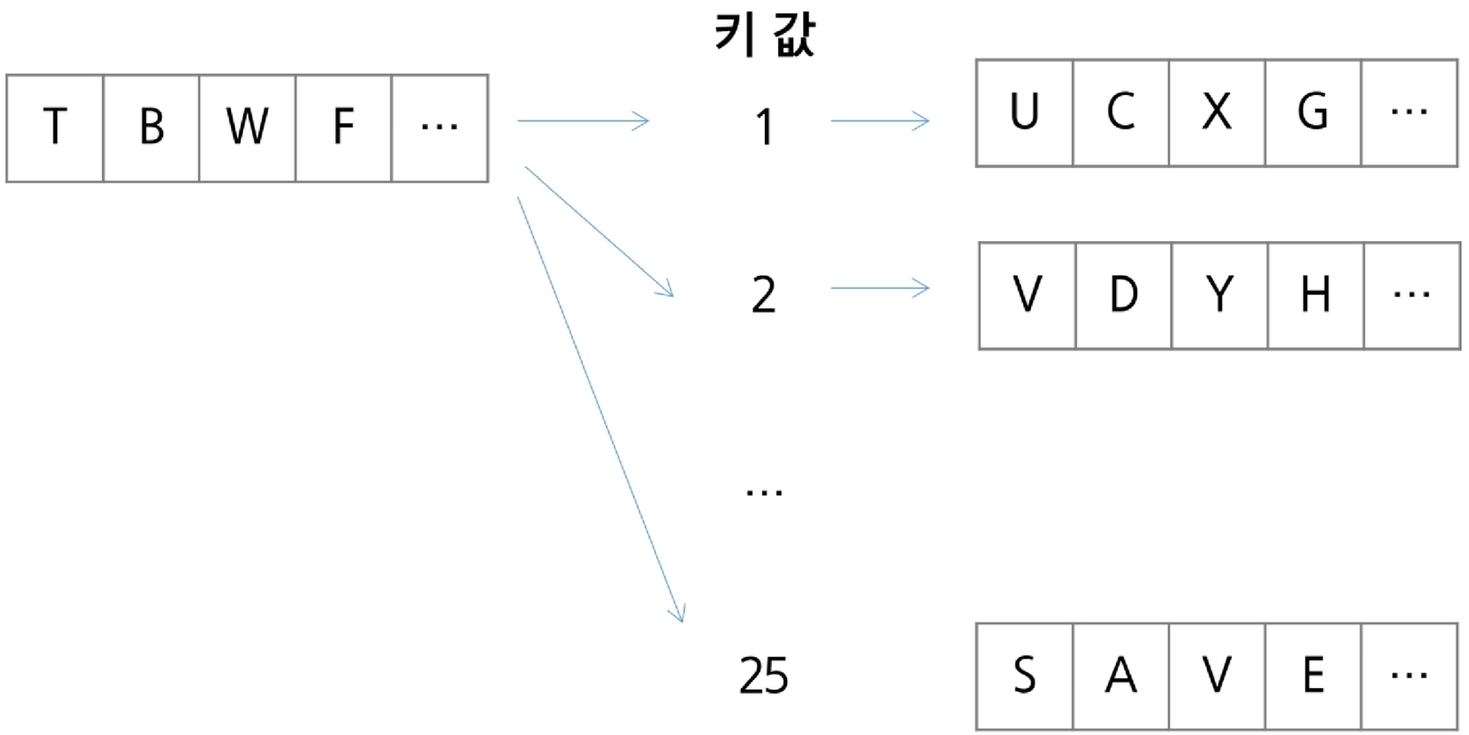
평 문	S	A	V	E		P	R	I	V	A	T	E		R	Y	A	N
암호문	T	B	W	F	A	Q	S	J	W	B	U	F	A	S	Z	B	O

<표3_SW문제해결 기본_String_시저 암호>

- 1만큼 평행했을 때 1을 키 값이라고 합니다.
- 수신자 이외의 사람(키가 1이라는 사실을 모르는 사람)이 암호문 TBWFAQSJWBUFASZBO을 보고 SAVE PRIVATE RYAN 라는 메시지를 맞출 수는 없을까요?

시저 암호문에 대한 전사공격

- 모든 키 값을 사용해 해석 시도
- 대문자만 사용한다는 것은 알고 있음



<그림18_SW문제해결 기본_String_시저 암호에 대한 전사공격>

문자 변환표를 이용한 암호화 (단일 치환 암호화)

- 단순한 카이사르 암호화보다 훨씬 강력한 암호화 기법
  - 알파벳 하나를 다른 고정된 알파벳으로 바꾸는 방법입니다.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Q	H	C	B	E	J	K	A	R	W	S	T	U	V	D		I	O	P	X	Z	F	G	L	M	N	Y

<표4_SW문제해결 기본_String_문자 변환표>

평 문	S	A	V	E		P	R	I	V	A	T	E		R	Y	A	N
암호문	X	H	G	J	Q	I	P	W	G	H	Z	J	Q	P	N	H	D

<표5_SW문제해결 기본_String_문자 변환표를 이용한 암호화>

단일 치환 암호화의 복호화

- 모든 키의 조합이 필요
- 키의 총 수

- 26 × 25 × 24 × 23 × ⋯ × 1 = 26! = 403291461126605635584000000
- 1초에 10억 개의 키를 적용할 수 있어도 120억년 이상 걸립니다.

- 빈도 분석 공격 가능

- 가장 많이 쓰이는 글자 E, T, A 등 입니다.
- 암호문에서 자주 나오는 문자를 추정해 대응시키면 해독이 가능합니다.

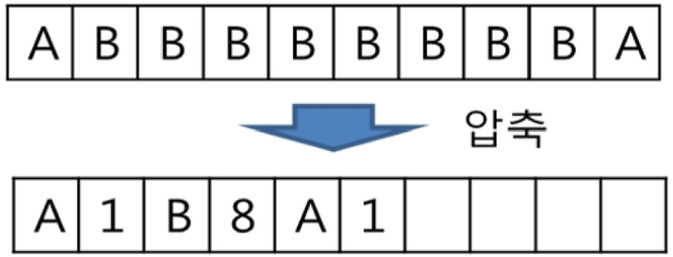
앨런 튜링(Alan Turing)은 기계식 암호 해독기 봄브(Bombe)를 개발해 제2차 세계대전 중에 독일군의 에니그마(Enigma) 암호를 해독했고, 이후 영국의 학자들이 로렌츠 암호를 해석하기 위해 세계 최초의 디지털 컴퓨터 콜로서스를 만든다. 존재가 기밀이어서 미국의 에니악이 오랫동안 최초의 컴퓨터로 알려져 있었다.

문자열 압축

- Run-length encoding 알고리즘

 - 같은 값이 몇 번 반복되는가를 나타내는 방식입니다.

 - 이미지 파일 포맷 중 BMP 파일의 압축 방법 중 하나로 사용합니다.



<그림19_SW문제해결 기본_String_Run-length encoding>

- 허프만 코딩 알고리즘

 - 더 효율적이고 일반적인 압축 방법입니다.

 - 자주 나오는 문자는 짧은 코드, 드물게 나오는 문자는 긴 이진 코드를 부여해 결과적으로 전체 데이터의 평균 비트 수를 최소화화는 알고리즘 입니다.

실습

문자열

- 4865. 글자수

회문

- 1989. 초심자의 회문 검사
- 4861. 회문

패턴 매칭

- 4864. 문자열 비교 (고지식한 방법으로)
- 3143. 가장 빠른 문자열 타이핑

추가 연습

- 5356. 의석이의 세로로 말해요
- 5432. 쇠막대기 자르기

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 문자열 자료형에 대한 설명으로 옳은 것은?

a) 리스트로만 표현할 수 있다.
b) 숫자와 동일한 연산자만 사용할 수 있다.
c) 변경 가능한(mutable) 자료형이다.
d) 문자열은 순서가 있는(ordered) 자료형이다.

2 UTF-8 인코딩에 대한 설명으로 옳지 않은 것은?

a) UTF-8은 가변 길이 바이트로 인코딩한다.
b) 영어 알파벳은 UTF-8에서 1바이트로 표현된다.
c) 전 세계 대부분의 문자와 호환된다.
d) 고정 길이 인코딩으로 4바이트로 표현한다.

3 다음 문자열 슬라이싱 결과는?

```
text = "Python"
print(text[1:4])
```

4 문자열 s = "apple"이 있을 때, s[-2]의 결과는?

a) 'p'
b) 'l'
c) 'e'
d) 'a'

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 문자열 replace() 메서드의 설명으로 바른 것은?

a) 대소문자를 자동으로 구분하지 않는다.
b) 원래 문자열을 직접 변경한다.
c) 새로운 문자열을 반환한다.
d) 문자열의 길이를 변경할 수 없다.

6 다음 중 문자열을 숫자로 변환할 수 없는 경우는?

a) "42"
b) "3.14"
c) " 123"
d) "3.14.15"

7 다음 설명에 맞는 메서드는?

문자열 "hello world"에서 'o' 문자가 처음 등장하는 인덱스를 반환하는 메서드는?

8 다음 설명에 맞는 메서드는?

"banana"라는 문자열에서 'a'의 개수를 세기 위한 메서드는?

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 다음 코드의 실행 결과는?

```
text = "hello python"
result = text.upper()
print(result)
```

10 다음 [ ]안에 들어갈 문자열을 역순으로 만드는 슬라이싱 표현식은?

```
text = "apple"
r_text = text[   ]
```

11 result에 저장된 값은?

```
text = " Hello   "
result = text.strip()
```

12 다음 코드의 실행 결과는?

```
text = '-'.join(["A", "B", "C"])
print(text)
```

확인문제

정답 및 해설

1. d) 문자열의 길이를 변경할 수 없다. 2. d) 고정 길이 인코딩으로 4바이트로 표현한다. 3. b) yth 4. b) 'l' 5. c) C. 새로운 문자열을 반환한다. 6. d) "3.14.15" 7. find 8. count 9. HELLO PYTHON 10. ::-1 11. Hello 12. A-B-C

1. 문자열은 인덱싱, 슬라이싱이 가능한 순서가 있는 자료형입니다. 하지만 변경은 불가능(immutable)합니다.

2. 4바이트 고정으로 인코딩하는 방식은 UTF-32 입니다.

3. 슬라이싱은 시작 인덱스 포함, 끝 인덱스 미포함입니다. 따라서 1~3까지 y, t, h가 출력됩니다.

4. 음수 인덱스는 뒤에서부터 셉니다. -1은 'e', -2는 'l' 입니다.

5. 문자열은 immutable이므로, replace()는 원래 문자열을 변경하지 않고 새로운 문자열을 반환합니다.

6. "3.14.15"는 숫자 형식이 잘못되어 float()나 int()로 변환할 수 없습니다.

7. find()는 특정 문자가 처음 등장하는 위치(인덱스)를 반환합니다. 찾지 못하면 -1을 반환합니다.

8. 문자열에서 특정 문자가 몇 번 나오는지 셀 때 count() 메서드를 사용합니다.

9. upper는 문자열의 모든 알파벳을 대문자로 바꿉니다.

10. 슬라이싱의 세 번째 인자(step)를 -1로 주면 역순이 됩니다.

11. strip()은 문자열의 앞뒤 공백을 제거합니다. 중간 공백은 제거하지 않습니다.

12. join()은 리스트의 각 요소 사이에 지정한 문자열을 삽입해 하나의 문자열로 만듭니다.

활동정리

핵심키워드

개념	설명	예시
코드체계	문자를 숫자로 표현하는 체계	ASCII, Unicode
문자열	문자들이 순서대로 나열된 데이터	‘hello’, “hello”, ‘’’hello’’’
문자열 연산	문자열에서의 덧셈(연결), 곱셈(반복) 등	‘hi’+’!’, ‘a’*5, ‘abc’ == ‘xyz’
KMP 알고리즘	효율적 문자열 검색	LPS 배열을 이용해 건너뛰기
보이어-무어 알고리즘	뒤에서부터 검사하는 빠른 검색 알고리즘	불일치, 접미어 휴리스틱 사용

<표6_SW문제해결 기본_String_핵심키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- **코드체계란?**
  - 문자에 대응되는 숫자를 정한 것
  - ASCII, Unicode로 규격화
  - 표준화되지 않은 부분 때문에 외부 인코딩이 필요

- **문자열**
  - 문자들이 순서대로 나열된 데이터
  - 순서가 구분되는 시퀀스 자료형이고 요소값을 변경할 수 없음(immutable)

- **연산**
  - 연결(+), 반복(*), 비교(==, is)등 다양한 연산 가능
  - 문자열 객체에 사용할 수 있는 다양한 메서드 제공

활동정리

오늘 공부한 내용 요약 및 정리

- **패턴 매칭**
  - 고지식한 방식은 비효율적이지만 구현할 수 있어야 함
  - KMP 알고리즘은 좀 더 빠르며, 보이어-무어 알고리즘은 상용 프로그램에서도 채택

- **암호화**
  - 단일 치환 암호화는 각 글자를 다른 글자로 바꾸는 것
  - 시저 암호 처럼 각 글자가 같은 키 값을 갖거나, 각 글자가 다른 키 값을 갖도록 문자변환표 이용

활동정리

웹페이지에서 ☒샖ᄇᆞ☒☒ ☒묎렐☒낅땲☒☒. 라는 문구가 표시됩니다.

1. 저장할 때 [ ]로 저장하고, 읽을 때 [ ]로 읽었기 때문입니다
2. 웹페이지라면 <meta charset="[ ]"> 지정이 필요합니다

활동정리

웹페이지에서 ☒샚ᄇᆞ☒☒ ☒묎렐☒낅땲☒☒. 라는 문구가 표시됩니다.

1. 저장할 때 UTF-8로 저장하고, 읽을 때 EUC-KR로 읽었기 때문입니다
2. 웹페이지라면 <meta charset="UTF-8"> 지정이 필요합니다

```
<meta charset="UTF-8">
<p>잘못된 접근입니다.</p>
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다